Guía de Estilo del Proyecto — EcoHarmony Park

Proyecto: TP6 — Ingeniería y Calidad de Software (ISW) · Grupo 1 · 4K1 · 2026 **Stack:** Node.js + TypeScript (backend, `node:http` + `node:sqlite`; mail con `nodemailer` + `qrcode`) · React 19 + Vite + Tailwind CSS v4 (frontend) · Vitest

> Esta guía describe **cómo está escrito el código hoy**. Cada regla sale de lo que ya hace la mayoría del repositorio, con ejemplos tomados de los archivos reales.

Índice

1. Principios Generales
2. Formato y Sangría
3. Convenciones de Nombres
4. Importaciones y Módulos
5. Funciones, Clases y Tipado
6. Constantes y Variables
7. Objetos y Arrays
8. Manejo de Errores
9. Comentarios
10. Componentes React (Frontend)
11. CSS y Estilos
12. Tests
13. Estructura del Proyecto
14. Herramientas de Automatización
15. Git y Ciclo TDD

---

1. Principios Generales

- **Las reglas del negocio viven en el backend.** El frontend importa las reglas, los precios y los tipos del backend por el alias `@backend/*`; nunca los copia.
- **Dominio en español, técnica en inglés.** Por ejemplo `ComprasService`, `SqliteUsuariosRepository`, `CompraPage`, `compras.types.ts`.
- **Inyección de dependencias.** Los servicios reciben sus colaboradores por constructor. Los servicios y repositorios se crean en un único lugar, aplicacion.ts; los adaptadores externos (pasarela y notificador) se eligen en main.ts según la configuración y se le pasan a `crearAplicacion`.
- **Responsabilidad única.** Un método público orquesta y delega en métodos privados con nombre de acción (`validarSolicitud`, `registrarCompra`, `iniciarPago`, `notificarCompra`).
- **Tipado estricto.** `"strict": true` en los dos `tsconfig.json` y ningún `any`.

---

2. Formato y Sangría

El formato lo decide **Prettier**, con .prettierrc.json, que comparten backend y frontend. Si `npm run format:check` pasa, el formato cumple la guía; no se revisa a mano.

Opción	Valor
Sangría	2 espacios, sin tabulaciones
Ancho de línea	100 caracteres
Punto y coma	Siempre
Comillas	Simples (`'`); dobles en los atributos JSX
Coma final	En todo lo que ocupa varias líneas

- Prettier no parte strings, template literals ni comentarios. Esas líneas pueden pasar de 100 caracteres y no cuentan como desvío.
- En el repositorio los archivos se guardan con **LF**. En Windows, con `core.autocrlf=true`, Git los convierte a CRLF en la copia local; eso es normal y no cuenta como desvío.

---

3. Convenciones de Nombres

Elemento	Estilo	Ejemplo real
Archivos de módulo del backend	`<modulo>.<rol>.ts`	`compras.service.ts`, `pagos.types.ts`, `pagos.constants.ts`, `compras.repository.ts`, `compras.controller.ts`
Adaptadores simulados	`<modulo>.<adaptador>-simulado.ts`	`pagos.pasarela-simulada.ts`, `compras.notificador-simulado.ts`
Adaptadores reales	`<modulo>.<adaptador>-<tecnología>.ts`	`compras.notificador-mail.ts`, `compras.transporte-smtp.ts`
Tipos que solo usa el servidor	`<modulo>.<tema>.types.ts`	`compras.mail.types.ts` (usa `Buffer`; el frontend no lo puede importar)
Infraestructura y arranque	`kebab-case.ts`	`servidor-http.ts`, `conexion-sqlite.ts`, `esquema-sqlite.ts`, `aplicacion.ts`, `main.ts`
Configuración	`<tema>.config.ts`	`base-datos.config.ts`, `servidor.config.ts`, `mail.config.ts`
Componentes React	`PascalCase.tsx`	`FichaVisita.tsx`, `PantallaResultado.tsx`
Páginas	`<Funcionalidad>Page.tsx`	`CompraPage.tsx`
Hooks	`use<Funcionalidad>.ts`	`useCompra.ts`
Utilidades del frontend	`camelCase.ts`	`formato.ts`, `cliente.ts`, `sesion.ts`

Elemento	Estilo	Ejemplo real
Tests	`<archivo probado>.test.ts`	`compras.service.test.ts`, `compras.repository.test.ts`
Tests de integración	`<modulo>[.<capa>].integracion.test.ts`	`compras.integracion.test.ts`, `compras.http.integracion.test.ts`
Fakes de test	`<puerto>.falso.ts`	`notificador.falso.ts`, `pasarela-pago.falsa.ts`, `transporte-mail.falso.ts`
Helpers de test	`kebab-case.ts` en `test/helpers/`	`leer-qr.ts`
Servicios	`<Modulo>Service`	`ComprasService`, `PagosService`, `ParqueService`
Repositorios	`Sqlite<Modulo>Repository`	`SqliteComprasRepository`
Controladores	`<Entidad>Controller`	`CompraController`
Adaptadores y fakes (clases)	`<Puerto>Simulada/o`, `<Puerto><Tecnología>`, `<Puerto>Falsa/o`	`PasarelaPagoSimulada`, `NotificadorMail`, `TransporteSmtp`, `NotificadorFalso`
Interfaces y tipos	`PascalCase`, **sin** prefijo `I`	`SolicitudCompra`, `PasarelaPago`, `ResultadoPago`
Props de componentes	`<Componente>Props`	`GuiaPasosProps`, `BotonProps`
Métodos y variables	`camelCase`	`validarFechaVisita`, `numeroOrden`
Constantes	`UPPER_SNAKE_CASE`	`MAX_ENTRADAS`, `PRECIO_ENTRADA_POR_PASE`, `DEMORA_REDIRECCION_MS`
Literales de dominio	`UPPER_SNAKE_CASE` entre comillas	`'VIP'`, `'EFECTIVO'`, `'REDIRIGIDO_A_MERCADO_PAGO'`
Columnas SQL	`snake_case`	`numero_orden`, `fecha_visita`, `tipo_pase`

Verbos de los métodos

El nombre empieza con un verbo en infinitivo que dice qué hace el método:

Verbo	Uso	Ejemplo
`validar…`	Lanza un error si la regla no se cumple	`validarFechaVisita`, `validarVisitantes`
`calcular…`	Hace un cálculo puro	`calcularPrecioEntrada`, `calcularDetalle`
`procesar…` / `iniciar…` / `enviar…`	Llama a un servicio externo	`procesarPago`, `iniciarPagoConMercadoPago`, `enviarConfirmacion`

Verbo	Uso	Ejemplo
`registrar…` / `guardar` / `contar…`	Persiste o consulta	`registrarCompra`, `guardar`, `contarEntradas`
`leer…`	Interpreta una entrada no confiable o lee la configuración	`leerSolicitud`, `leerEdad`, `leerConfiguracionServidor`
`crear…`	Fábrica	`crearAplicacion`, `crearServidor`, `crearDependencias`
`formatear…` / `a…`	Hace una conversión de presentación	`formatearPesos`, `aFechaIso`, `aVisitante`
`armar…`	Compone una estructura de salida a partir de datos	`armarMailConfirmacion`, `armarHtml`, `armarFilaDato`
`generar…`	Produce un código o una imagen	`generarCodigoQR`, `generarImagenQR`

Booleanos

- **En el backend, en los helpers y en las variables locales de la lógica del hook** se escriben como pregunta: `estaAbierto`, `estaRegistrado`, `esFeriado`, `esLunes`, `pagaMediaEntrada`, `hayAnterior`, `haySiguiente`, `puedeQuitar`, `esExitosa`, `estanCompletos`.
- **En el estado de React (`useState`, `useRef`), en las props y en las variables locales del render de un componente** se aceptan adjetivos o participios que se leen naturalmente en JSX: `abierta`, `elegida`, `enviando`, `aprobado`, `mailVisible`, `ancho`, `primerRender`.

Callbacks y acciones en React

- Las props que reciben callbacks llevan el prefijo **`al`**: `alElegir`, `alCerrar`, `alVolver`, `alCambiar`, `alQuitar`, `alComprarOtra`.
- Las acciones que expone el hook son verbos del dominio: `continuar`, `retroceder`, `agregarVisitante`, `elegirFormaPago`, `volverAEmpezar`. No se usa el prefijo `handle`.
- Los setters de `useState` se exponen con su nombre original: `setFecha`, `setEtapa`.

```
// ✅ Correcto
<PantallaResultado resultado={compra.resultado} fecha={compra.fecha}
alComprarOtra={compra.volverAEmpezar} />

// ❌ Incorrecto
<PantallaResultado onComprarOtra={handleComprarOtra} />
```

---

4. Importaciones y Módulos

- Se usan **ES Modules** en todo el proyecto (`"type": "module"`).
- Los imports que solo traen tipos usan **`import type`**.

- **Rutas:**
  - En el backend, las rutas son relativas y van sin extensión.
  - En el frontend, lo que viene del backend se importa con el alias `@backend/...` y lo propio con rutas relativas.
- **Exports con nombre en todos los archivos.** Solo los archivos de configuración de las herramientas (`vite.config.ts`, `vitest.config.ts`) usan `export default`, porque la herramienta lo exige.

```
// ✅ Correcto (useCompra.ts)
import { useState } from 'react';
import type { ResultadoCompra, SolicitudCompra } from
'@backend/modules/compras/compras.types';
import { ParqueService } from '@backend/modules/parque/parque.service';
import { ErrorConexion, postJson } from '../../shared/http/cliente';

// ✅ Correcto
export const CompraPage = () => { ... };

// ❌ Incorrecto
const CompraPage = () => { ... };
export default CompraPage;
```

---

5. Funciones, Clases y Tipado

Clases

Los servicios, repositorios, controladores y adaptadores son **clases**. Los colaboradores llegan por constructor como `private readonly`.

```
// ✅ Correcto
export class PagosService {
  constructor(private readonly pasarela: PasarelaPago) {}
}

export class ComprasService {
  constructor(private readonly deps: DependenciasCompras) {}
}
```

Las clases no crean a sus colaboradores. El único lugar donde se hace `new` de servicios y repositorios es `crearAplicacion` (aplicacion.ts). Los adaptadores externos se arman en `main.ts`: `PasarelaPagoSimulada` y, si hay `SMTP_HOST`, `NotificadorMail` → `TransporteSmtp` → nodemailer; si no, `NotificadorSimulado`.

Funciones sueltas

- Los helpers de módulo, los componentes y los hooks son **arrow functions** asignadas a `const`.
- Si el cuerpo es una sola expresión, se usa la forma concisa.
- El parámetro **siempre** va entre paréntesis, aunque sea uno solo.

```
// ✅ Correcto
const esRegistro = (valor: unknown): valor is Record<string, unknown> =>
  typeof valor === 'object' && valor !== null && !Array.isArray(valor);

// ❌ Incorrecto
const esRegistro = valor => typeof valor === 'object';
```

Tipos de retorno

- Los **métodos públicos y privados de las clases** declaran siempre su tipo de retorno, por ejemplo `Promise<ResultadoPago>`, `void` o `boolean`.
- Las **funciones exportadas de utilidades** también lo declaran: `formatearPesos(importe: number): string`.
- En los componentes React, en los hooks y en los helpers internos de un archivo se acepta el tipo inferido. En `useCompra`, además, su tipo inferido es el que define `EstadoCompra`.

Validar primero

Las validaciones van al principio y terminan antes de modificar estado o llamar a un servicio externo.

```
// ✅ Correcto (compras.service.ts)
async comprarEntradas(solicitud: SolicitudCompra): Promise<ResultadoCompra> {
  await this.validarSolicitud(solicitud);

  const detalle = this.deps.pagos.calcularDetalle(visitantes);
  const numeroOrden = await this.registrarCompra({ ... });
  const pago = await this.iniciarPago(numeroOrden, detalle.total, formaPago);
  await this.notificarCompra(email, { ... });
  ...
}
```

En una compra, el orden es: **validar, calcular, registrar, pagar, notificar**. Dentro de la validación, el orden es: fecha, usuario, forma de pago, cantidad y visitantes.

Tipado estricto

- **Nunca se usa `any`.** Lo que llega de afuera (el body HTTP, una respuesta `fetch`) se tipa como `unknown` y se acota con *type guards* (`esRegistro`, `leerSolicitud`, `leerMensajeError`).
- En el código de producción se usa `as` en solo dos casos:
  - después de haber validado la forma, como en `cuerpo.visitantes as SolicitudCompra['visitantes']`;
  - para tipar una fila de SQLite: `.get(...) as { cantidad: number }`.
- En los tests, `as` también se acepta en dos casos:
  - para forzar a propósito un valor inválido (`'CHEQUE' as FormaPago`);
  - para tipar la respuesta de `fetch` (`await respuesta.json() as { … }`).
- Los estados posibles se modelan con **uniones de literales** y **uniones discriminadas**:

```
export type ResultadoPago =
  | { formaPago: 'EFECTIVO'; estado: 'PENDIENTE_EN_BOLETERIA' }
  | { formaPago: 'TARJETA'; estado: 'REDIRIGIDO_A_MERCADO_PAGO'; urlRedireccion:
string };
```

Fechas

- **Fecha de visita:** es un `Date` local a medianoche. En la API viaja como texto `AAAA-MM-DD`. La conversión se hace en los bordes:
  - en el controller, con `leerFechaLocal`;
  - en el frontend, con `aFechaIso`;
  - en SQL, con `formatearFechaSql`;
  - en el mail, con `formatearFecha` de `compras.mail-confirmacion.ts`, que la arma en UTC a partir de sus partes locales para que no se corra un día cuando el proceso corre en UTC (Docker).
- **Fecha actual:**
  - `ParqueService.validarFechaVisita(fechaVisita, ahora)` recibe `ahora` como parámetro. Si no se pasa, toma `new Date()` por defecto.
  - Los tests **siempre** pasan `ahora` en forma explícita.
  - En el frontend, el reloj se lee una sola vez en `useCompra` y se pasa hacia abajo.
- **Zona horaria:** las reglas horarias se evalúan en `America/Argentina/Cordoba`, sin importar la zona del proceso. `ParqueService` lee el año, el mes, el día, la hora y el minuto de `ahora` con `Intl.DateTimeFormat` (`partesEnCordoba`). Por eso da igual que el back corra en UTC (Docker) o que el navegador esté en otra zona.

Dinero

- Los importes son **enteros en pesos argentinos (ARS)**.
- En la base se guardan como columnas `INTEGER CHECK (... >= 0)`.
- Los descuentos se aplican con `FACTOR_DESCUENTO_MEDIA_ENTRADA` sobre el precio del pase.
- En el frontend y en el mail se muestran con `Intl.NumberFormat('es-AR', { currency: 'ARS', maximumFractionDigits: 0 })`. Una entrada de $0 se muestra como «Gratis»; el total se muestra siempre en pesos.

---

6. Constantes y Variables

- Se usa **`const`** por defecto. `let` queda solo para acumuladores o asignaciones dentro de un `try` (`precioTotal`, `respuesta`, `json`). No se usa `var`.
- **Todo valor del negocio es una constante con nombre**, en `UPPER_SNAKE_CASE`:
  - Si se comparte entre módulos o con el front, va en `<modulo>.constants.ts`, con un comentario que aclara la unidad o el criterio:

```
// Precio completo de la entrada (en ARS) según el tipo de pase.
export const PRECIO_ENTRADA_POR_PASE: Record<TipoPase, number> = {
  VIP: 20000,
```

```
  REGULAR: 10000,
};

// Límites de edad inclusivos para cada beneficio.
export const EDAD_MAX_NINOS_GRATIS = 3;
```

  - Si es privada de un módulo, va al principio del archivo (`const MAX_ENTRADAS = 10;`).
  - Si es privada de una clase, va como `private static readonly` (`ParqueService.HORA_LIMITE`).

- Las **tablas de lookup** también van en mayúsculas: `ACCION`, `NOMBRE_PASE`, `ESTILOS`, `PASES`, `DIAS_SEMANA`, `MENSAJES_DE_REGLA`.
- **Los prefijos del dominio** son constantes en `compras.constants.ts`, con un comentario que muestra el formato completo: `PREFIJO_ORDEN = 'ORD-'` (`ORD-<uuid>`) y `PREFIJO_QR = 'QR-'` (`QR-<numero de orden>-<n>`, un código por entrada).
- **Configuración de ejecución:** está en un único objeto `configuracion` (`as const`) dentro de config/configuracion.ts y se lee con `leerConfiguracionBaseDatos()`, `leerConfiguracionServidor()`, `leerConfiguracionSimulacion()` y `leerConfiguracionMail()`.
- **Valores de cada máquina** (credenciales SMTP, remitente, `EMAIL_DEMO`): se leen de `process.env`. Vienen del `.env` de la raíz, que no se commitea; la plantilla versionada es `.env.example`. `npm start` lo carga con `--env-file-if-exists` y Docker con `env_file`. Sin `.env`, todo sigue funcionando con los valores por defecto y el mail simulado.
- **No cuentan como números mágicos:**
  - los códigos HTTP (`201`, `400`, `422`, `500`);
  - los valores de Tailwind entre corchetes (`text-[15px]`);
  - el `+ 1` / `- 1` para convertir el mes de `Date`.

---

7. Objetos y Arrays

- Lleva **coma final** todo lo que ocupa varias líneas.
- Se usa **destructuring** para sacar varias propiedades: `const { email, fechaVisita, cantidad, visitantes, formaPago } = solicitud;`
- Se usan **shorthand properties**: `return { numeroOrden, cantidadEntradas: cantidad, fechaVisita, ... }`.
- **Actualizaciones inmutables** en el estado de React: `lista.map((v) => (v.id === id ? { ...v, ...cambios } : v))` y `[...lista, nuevoVisitante()]`.
- Las tablas indexadas por una unión se tipan con `Record<Union, T>`, y los mapas de literales usan `as const`.
- **Recorridos:**
  - En el frontend se prefieren `map`, `filter` y `flatMap`.
  - En el backend se usa `for...of` cuando el recorrido tiene efectos (insertar filas, acumular).

---

8. Manejo de Errores

Backend

- **Las reglas lanzan errores.** Una validación hace `throw new Error('<mensaje en español con punto final>.')`; nunca devuelve `false` ni `null`.
- **El texto del mensaje es parte del contrato.** El controller traduce:

Situación	Respuesta
Body que no se puede interpretar	`400 { error: 'Solicitud inválida.' }`
Mensaje incluido en `MENSAJES_DE_REGLA`	`422 { error: <mensaje> }`
Cualquier otro error	`500 { error: 'Error interno.' }`, y el error se registra con `console.error` en el servidor
Ruta inexistente	`404 { error: 'No encontrado.' }`

  Si se agrega una regla nueva, su mensaje se suma a `MENSAJES_DE_REGLA` **en el mismo PR**.

- **Fallas externas:** se envuelven en un mensaje de dominio, por ejemplo `'No se pudo iniciar el pago con Mercado Pago.'`. El envío del mail es la excepción: si el SMTP falla, su error llega tal cual al controller, que responde 500 y lo deja en el log. La compra ya quedó registrada.
- Si el error no se usa, el `catch` va sin variable: `} catch {`.
- **Transacciones:** se usa `BEGIN` / `COMMIT`, y en el `catch` se hace `ROLLBACK` y se vuelve a lanzar el error.
- **Tono de los mensajes:** neutro e impersonal. Por ejemplo `'Debe seleccionar una forma de pago.'` o `'No se pueden comprar más de 10 entradas.'`.

Frontend

- **Errores de red:** se convierten en `ErrorConexion`, que ya trae un mensaje para mostrar.
- **Errores de la API:** se muestra el texto de `{ error }` dentro de `<Alerta>`.
- **Errores de un campo:** se muestran con `<MensajeCampo id=…>`, enlazado al campo por `aria-describedby`.
- **Tono:** voseo e imperativo, por ejemplo `'Elegí una fecha de visita.'`, `'Ingresá la edad.'`, `'Elegí un pase.'`.
- **Cuándo aparecen:** los errores de cada paso se muestran recién después del primer intento de avanzar (`intentoAvanzar`).
- **No se dejan `catch` vacíos.** Si se ignora un error, el bloque asigna un valor por defecto, como `json = undefined`.

---

9. Comentarios

- Se escriben **en español** y explican el **por qué**, no el qué.
- **Formato:**
  - `//` arriba de la línea o del bloque que se explica;
  - `/** … */` en los helpers exportados y en las props de los componentes compartidos.

- **Referencias:** se cita la fuente de la regla cuando corresponde, por ejemplo `FAQ 4`, `FAQ 5`, `FAQ 19` o `CONVENCIONES.md §1.1`.
- **Los comentarios tienen que coincidir con el código.** Un comentario que dice otra fecha u otro valor que el código es un error.
- No se deja código comentado.

```
// ✅ Correcto: explica el por qué
// El backend no expone CORS: en desarrollo las llamadas a /api pasan por el proxy
de Vite.

/** Edad como número entero, o `undefined` si el campo no tiene un número. */
export const leerEdad = (texto: string): number | undefined => ...

// ❌ Incorrecto: describe lo obvio
// Se crea la variable total
const total = detalle.total;
```

---

10. Componentes React (Frontend)

Estructura de un archivo de componente

```
// 1. Importaciones (externas, después internas)
import { useEffect, useRef, useState } from 'react';
import { Boton } from '../../../shared/ui/Boton';

// 2. Constantes del módulo
const DEMORA_REDIRECCION_MS = 1400;

// 3. Props: tipo en línea si son pocas, interface <Componente>Props si son más
interface PantallaResultadoProps {
  resultado: ResultadoCompraHttp;
  fecha: Date;
  alComprarOtra: () => void;
}

// 4. Componente exportado como arrow function con export con nombre
export const PantallaResultado = ({ resultado, fecha, alComprarOtra }:
PantallaResultadoProps) => {
  // 5. Hooks al principio
  const [mailVisible, setMailVisible] = useState(true);
  const titulo = useRef<HTMLHeadingElement>(null);

  // 6. Valores derivados
  const entradas = `${cantidad} ${cantidad === 1 ? 'entrada' : 'entradas'}`;

  // 7. Efectos, siempre con su array de dependencias
  useEffect(() => { ... }, [mailVisible]);
```

```
  // 8. JSX
  return ( ... );
};

// 9. Subcomponentes privados del archivo, debajo y sin export
const Renglon = ({ termino, children }: { termino: string; children: ReactNode })
=> ( ... );
```

Estado y lógica

- El estado de la compra vive en **un único hook**, `useCompra`. Devuelve un objeto con el estado, los valores derivados y las acciones. Su tipo es `EstadoCompra = ReturnType<typeof useCompra>`.
- La página pasa ese objeto completo a cada paso (`<PasoFecha compra={compra} />`), y los pasos no guardan estado de negocio propio.
- **Las reglas no se reescriben en el front:**
  - la validez de una fecha sale de `ParqueService`;
  - los precios salen de `PagosService.calcularDetalle`;
  - los textos de descuento salen de `pagos.constants.ts`.

JSX

- Hay un solo elemento raíz; si no hace falta un contenedor, se usa `<>…</>`.
- Para armar `className` con condiciones se usa un template literal o `[...].join(' ')`.
- Todo `<button>` que no envía un formulario lleva `type="button"`. La variante estándar es el componente `<Boton>`.
- Los íconos son solo de `lucide-react`. Los decorativos se ocultan a los lectores de pantalla con `aria-hidden`, puesto en el ícono o en el elemento que lo contiene.
- La única fuente de estilos son las clases de Tailwind. Se acepta `style` solo dentro de un SVG (`Sello`).

Accesibilidad (obligatoria)

- **Títulos:** cada pantalla tiene un `<h1>`. Al cambiar de paso, el foco pasa a ese título (`tabIndex={-1}`).
- **Controles:**
  - Los inputs son nativos; los radios usan `sr-only` dentro de un `<label>` visual.
  - Los grupos van en `<fieldset>` con `<legend>` (puede ser `sr-only`).
  - Los botones sin texto llevan `aria-label` (`'Quitar visitante 2'`).
- **Campos inválidos:** los inputs y los `radiogroup` llevan `aria-invalid` y `aria-describedby`, que apunta al id del `MensajeCampo`. Los grupos compuestos con `role="group"`, como el calendario, llevan solo `aria-describedby`, porque ARIA no admite `aria-invalid` en ese rol.
- **Valores que cambian solos** (totales, mes del calendario): llevan `aria-live="polite"`.
- **Modales:** se usa `<dialog>` nativo con `showModal()`.
- **Tamaño táctil:** los objetivos miden 44px o más (`h-11`, `size-11` o más). En grillas densas, como los días del calendario en un celular angosto, se acepta un mínimo de 24px (WCAG 2.2 AA).

Textos de la interfaz

- **Voseo rioplatense:** «Elegí», «Tocá», «Pagás».

- **Títulos de los pasos:** se escriben como preguntas, por ejemplo «¿Qué día venís?», «¿Quiénes vienen?», «¿Cómo pagás?».
- **Botones:** dicen la acción, por ejemplo «Continuar», «Continuar al pago», «Confirmar compra».

Pago simulado y mail

- **Pago con tarjeta:** pasa por `PantallaMercadoPago`, que espera `DEMORA_REDIRECCION_MS` y muestra «Compra exitosa».
- **Pago en efectivo:** va directo a `PantallaResultado`.
- **Mail:** lo envía el backend al confirmar la compra, con un QR por entrada. `ModalMail` se abre al llegar al resultado y confirma la casilla del usuario de la sesión simulada (`USUARIO_SESION`).
- **Usuario de la sesión:** `USUARIO_SESION.email` es `EMAIL_DEMO`, que `vite.config.ts` inyecta en el build (`import.meta.env.EMAIL_DEMO`, tipado en `vite-env.d.ts`). Es el mismo usuario que el backend registra al arrancar, así front y back no se desincronizan.

---

11. CSS y Estilos

Tecnología

- Se usa **Tailwind CSS v4**, con utilidades en `className`. No hay BEM, CSS Modules ni archivos CSS por componente.
- Los tokens se definen **una sola vez** en `@theme` dentro de src/shared/styles/index.css. Tailwind los publica como variables en `:root` y genera las utilidades (`bg-primario`, `text-alerta`, `border-renglon`).
- **En los componentes no se escriben colores hexadecimales ni `rgb()`.**
  - Para transparencias se usa el modificador de opacidad (`text-superficie/85`, `border-tinta-suave/50`).
  - Para sombras se usa `color-mix(in_srgb,var(--color-tinta)_12%,transparent)`.

Paleta

Token	Hex	Uso en el código
`primario`	`#386641`	Fondo del header, títulos (`h1`/`h2`), total, botón principal con texto blanco, borde de todo lo elegido, filete superior de las hojas, íconos informativos
`secundario`	`#6a994e`	Hover y active del botón principal; borde del botón secundario y, con opacidad (`secundario/20`, `/30`), su hover y active; spinner; borde punteado del lugar del sello
`acento`	`#a7c957`	Solo como relleno de selección (día, pase, forma de pago, `::selection`, disco del ícono del modal), siempre con borde `primario` o dentro de un contenedor con ese borde. El texto encima va en `tinta`. Nunca es color de texto
`fondo`	`#f2e8cf`	Fondo de `body` y hover de filas, opciones, días del calendario y botones sin borde
`alerta`	`#bc4749`	Solo errores: `Alerta`, `MensajeCampo`, borde o `ring` de campos inválidos, hover del botón de quitar visitante

Token	Hex	Uso en el código
`superficie`	`#ffffff`	Hojas, ficha, modal, fondo de los mensajes de error y texto sobre `primario`
`renglon`	`#e3d8ba`	*Derivado.* Filetes de 1px entre filas y bordes de los días disponibles
`tinta`	`#1f3a25`	*Derivado.* Texto de cuerpo
`tinta-suave`	`#4b5a44`	*Derivado.* Texto secundario, etiquetas, términos de `<dt>`, días no disponibles (con opacidad)

Hay tres excepciones a "sin hex fuera de `@theme`":

- `<meta name="theme-color" content="#386641">` en `index.html`, porque ahí no se pueden usar variables CSS;
- el fondo de `dialog::backdrop`, que usa `rgb()` con los valores de `tinta`;
- el HTML del mail de confirmación (`compras.mail-confirmacion.ts`). Los clientes de correo ignoran `<style>`, las clases y las variables CSS, así que el mail repite la paleta en la constante `COLOR`, usa estilos inline y se maqueta con tablas. Respeta los mismos usos de la tabla de arriba y la escala tipográfica de abajo, con la fuente `Lato, 'Helvetica Neue', Arial, sans-serif`.

Tipografía

- **Fuente:** Lato, cargada con `@fontsource/lato` (400 y 700), con `--font-sans: Lato, system-ui, sans-serif`.
- **Pesos:** solo `font-normal` (400) y `font-bold` (700).
- **Escala en uso** (no se agregan tamaños fuera de esta tabla):

Tamaño	Clase	Uso
11px	`text-[11px]`	Solo el texto «sin sellar» del lugar del sello
12px	`text-xs`	Iniciales de los días de la semana del calendario
13px	`text-[13px]`	Etiquetas de campo, notas, número de orden, subtítulo del header
14px	`text-sm`	Leyendas, contador de entradas, tarifas de referencia, email del header, botones de texto chicos
15px	`text-[15px]`	Cuerpo
16px	`text-base`	Números de los días del calendario
17px	`text-[17px]`	Cuerpo destacado, inputs y marca del header
18px	`text-lg`	Mes del calendario
19px	`text-[19px]`	Botón principal
20px	`text-xl`	Título de la pantalla de espera de Mercado Pago
22px	`text-[22px]`	Título del modal, total de la ficha final

Tamaño	Clase	Uso
26px	`text-[26px]`	Título de paso, total y «Compra exitosa»
28px	`text-[28px]`	Título de resultado
30 / 36px	`lg:text-3xl` / `lg:text-4xl`	Títulos de paso y de resultado desde `lg`

- **Cifras:** los importes que se muestran sueltos (renglones, subtotales, totales) llevan la utilidad `cifras` (números tabulares). Dentro de una oración no hace falta.

Forma, movimiento y layout

- **Bordes:** `rounded-md` para hojas, botones e inputs; `rounded-full` para círculos.
- **Sombras:** solo en lo que flota, es decir la ficha fija en el celular y el modal.
- **Transiciones:** cortas, para cambios de color o el giro de un ícono. No hace falta desactivarlas.
- **Animaciones:** lo que se desplaza por la pantalla (el sello y el camino de la guía de pasos) se desactiva con `prefers-reduced-motion`.
- **Mobile first:** las clases base son para el celular, y se amplía con `sm:` y `lg:`.
  - El contenedor estándar es `mx-auto max-w-xl px-4 lg:max-w-5xl`.
  - En el celular la ficha de visita queda fija al pie; desde `lg` pasa a una columna lateral.
- **Utilidades propias:** se declaran con `@utility` en `index.css`, como `cifras` y `renglon-arriba`.

Calendario (FAQ 2 y 25)

- **Días cerrados** (lunes, 25/12 y 1/1): se muestran **tachados** y deshabilitados.
- **Días fuera de rango** (antes de hoy, después de un mes, o hoy después de las 17:30): se muestran **atenuados** y deshabilitados.
- **El motivo** sale del mensaje de `ParqueService` y se expone en `aria-label` y en `title`.
- **La semana empieza el lunes.** Solo se puede navegar a meses que tengan algún día a la venta.

---

12. Tests

Herramientas y ubicación

- Se usa **Vitest**. Corren todos los `backend/test/**/*.test.ts` con `npm test`.
- **Tests unitarios de los tres servicios:** van en la raíz de `test/`, por ejemplo `parque.service.test.ts`, `pagos.service.test.ts` y `compras.service.test.ts`.
- **Tests de infraestructura y adaptadores:** repiten la ruta de `src/` bajo `test/`, por ejemplo `test/modules/compras/compras.repository.test.ts` o `test/infraestructura/bd/conexion-sqlite.test.ts`.
- **Tests de integración:**
  - `test/compras.integracion.test.ts`: escenarios de usuario con los servicios reales;
  - `test/modules/compras/compras.http.integracion.test.ts`: HTTP más SQLite en memoria. Uno de sus tests usa el `NotificadorMail` real con `TransporteMailFalso` y decodifica los QR del mail.
- **Fakes reutilizables:** van en `test/fakes/`. Registran sus llamadas en `llamadas: […]`.

- **Helpers compartidos que no son fakes:** van en `test/helpers/`, como `leerQR`, que decodifica un PNG con `jsqr` y `pngjs` para comprobar que un QR se puede leer.
- El frontend no tiene tests.
- **Cobertura:** `npm run test:coverage` mide los tres servicios y falla por debajo del 90 % de líneas.

Dobles de prueba

Qué se prueba	Cómo se reemplazan las dependencias
Servicios	`vi.fn<Firma>()` tipado, con `mockResolvedValue`/`mockReturnValue`
Repositorios y conexión	SQLite real en memoria (`new ConexionSqlite(':memory:')`)
Controller	Servicio falso con `implements Pick<ComprasService, 'comprarEntradas'>` y servidor real en el puerto `0`
Integración	Servicios reales. Pasarela, notificador y repositorios falsos o en memoria
`NotificadorMail`	`TransporteMailFalso`. Los QR y el armado del mail son reales
`TransporteSmtp`	Transportador real de nodemailer con `jsonTransport`: arma el mail sin salir a la red
Código que escribe en la consola	`vi.spyOn(console, 'error').mockImplementation(() => {})`, restaurado con `vi.restoreAllMocks()` en `afterEach`

Nombres

- **`describe` en los tests de servicios:** primero la clase o la US y adentro el método, por ejemplo `describe('ParqueService')` y dentro `describe('validarFechaVisita')`. Todos los `describe` del archivo cuelgan de esa raíz. Si el archivo prueba un solo método, la raíz puede ser `Clase.método` y adentro agrupar por tema (`'compra exitosa'`, `'validaciones'`).
- **`describe` en los tests de infraestructura** (repositorios, conexión, controller, adaptadores): alcanza con un `describe` por clase.
- **`it`:** una frase en presente que describe el comportamiento observable, por ejemplo `'rechaza la compra de más de 10 entradas'`, `'acepta el día actual si se compra exactamente a las 17:30'` o `'debe rechazar una edad negativa'`. Se aceptan las dos formas, con «debe» o sin él.
- **Escenarios de usuario:** el nombre termina en `(pasa)` o `(falla)`.

Estructura del test

- **Arrange, Act y Assert van separados por una línea en blanco**, sin comentarios. Cuando el Act es una excepción, Act y Assert se juntan en `expect(() => …).toThrow(…)`.
- En los casos de error de `ComprasService` y de los tests de integración, además del mensaje se verifica que **no hubo efectos**: `expect(deps.compras.guardar).not.toHaveBeenCalled()`.
- Para tablas de casos se usa `it.each`, con la tupla tipada:

```
it.each<[number, TipoPase, number]>([
  [30, 'VIP', 20000],
  [3, 'REGULAR', 0],
  [60, 'VIP', 10000],
```

```
])('debe un visitante de %i años con pase %s pagar $%i', (edad, tipoPase,
precioEsperado) => {
  expect(pagos.calcularPrecioEntrada(edad, tipoPase)).toBe(precioEsperado);
});
```

Datos de prueba

- **Constantes del test:** van en mayúsculas al principio del archivo (`EMAIL`, `FECHA_VISITA`, `URL_MP`, `DETALLE`).
- **Builders:**
  - `crearDependencias()` arma dependencias nuevas en cada test;
  - `solicitudValida(cambios)` arma una solicitud válida a la que se le pisan solo los campos necesarios;
  - `visitantes(n)` y `crearSistema()` arman listas y sistemas completos.
- **Fechas:**
  - En `parque.service.test.ts` se usan dos helpers: `fecha(anio, mes 1-12, dia)` para la fecha de visita y `enCordoba(anio, mes, dia, hora, minuto)` para el instante `ahora`, armado en UTC-3.
  - Para probar que una regla no depende de la zona del proceso, el `describe` fija `process.env.TZ = 'UTC'` en `beforeEach` y lo restaura en `afterEach`.
  - En el resto se usa `new Date(a, m - 1, d)`.
  - Para fijar "hoy" en integración: `vi.useFakeTimers({ toFake: ['Date'] })` más `vi.setSystemTime`.
- **Recursos:** se cierran en `afterEach` (`conexion.cerrar()`, `servidor.close()`).
- **Omitir un campo obligatorio a propósito:** se usa `// @ts-expect-error: <motivo>`.

---

13. Estructura del Proyecto

```
ISW_TP6/
├── CONVENCIONES.md                 ← Convenciones acordadas con el PO y la
cátedra
├── guia_estilo_tp6.md              ← Este documento
├── .prettierrc.json / .oxlintrc.json ← Formato y lint compartidos por backend y
frontend
├── .env.example                    ← Plantilla del .env (EMAIL_DEMO y
credenciales SMTP)
├── .gitignore                      ← Ignora el .env de la raíz
├── docker-compose.yml              ← Backend + frontend (nginx) con
healthchecks; pasa el .env
├── backend/
│   ├── Dockerfile                  ← Imagen de la API (bundle ESM con
createRequire para qrcode y nodemailer)
│   ├── bd/data/                    ← Archivo SQLite local (ignorado por git)
│   ├── src/
│   │   ├── main.ts                 ← Arranque: lee config, elige adaptadores,
carga datos iniciales, levanta HTTP
│   │   ├── aplicacion.ts           ← Raíz de composición: crea y conecta
servicios y repositorios
```

```
│   │   ├── servidor-http.ts        ← Servidor node:http (POST /api/compras)
│   │   ├── config/                 ← configuracion.ts + lectores
<tema>.config.ts (base de datos, servidor, simulación, mail)
│   │   ├── infraestructura/bd/     ← Conexión, esquema DDL y datos iniciales de
SQLite
│   │   └── modules/
│   │       ├── compras/            ← service, types, constants, repository,
controller, notificador-simulado,
│   │       │                         y el mail: qr, mail-confirmacion,
mail.types, notificador-mail, transporte-smtp
│   │       ├── pagos/              ← service, types, constants, pasarela-
simulada
│   │       ├── parque/             ← service
│   │       └── usuarios/           ← repository
│   └── test/
│       ├── <modulo>.service.test.ts ← Unitarios de los tres servicios
│       ├── compras.integracion.test.ts
│       ├── fakes/                  ← Fakes reutilizables
│       ├── helpers/                ← Helpers de test (leer-qr.ts)
│       ├── infraestructura/bd/     ← Igual que src/
│       └── modules/<modulo>/       ← Igual que src/
└── frontend/
    ├── Dockerfile                  ← Build de Vite servido por nginx
    ├── nginx.conf                  ← Sirve el build y hace de proxy de /api al
backend
    ├── index.html
    ├── public/logo-ecoharmony.png  ← Logo provisorio (pendiente de validación
del PO, FAQ 17)
    ├── vite.config.ts              ← Alias @backend, proxy /api → :3000 e
inyección de EMAIL_DEMO
    └── src/
        ├── main.tsx                ← Punto de entrada React
        ├── vite-env.d.ts           ← Tipo de import.meta.env.EMAIL_DEMO
        ├── app/App.tsx             ← Shell: header y <main>
        ├── modules/compra/
        │   ├── CompraPage.tsx      ← Página y navegación entre pasos y pantallas
        │   ├── useCompra.ts        ← Estado y acciones de la compra
        │   └── components/         ← Pasos, ficha, pantallas de resultado, modal
y sello
        └── shared/
            ├── http/cliente.ts     ← Llamadas a la API (postJson, ErrorConexion)
            ├── sesion/sesion.ts    ← Usuario de la sesión simulada (EMAIL_DEMO)
            ├── styles/index.css    ← Tokens @theme y estilos base
            └── ui/                 ← Componentes y formatos reutilizables
(Boton, Alerta, Calendario, formato)
```

Principios de organización

- **Backend por módulo de dominio.** Cada módulo agrupa sus roles con el patrón `<modulo>.<rol>.ts`. Lo transversal (configuración, infraestructura, arranque) queda fuera de `modules/`.
- **Frontend por funcionalidad.** Cada funcionalidad tiene su carpeta en `modules/`, con su página, su hook y sus componentes. Lo que se reutiliza entre funcionalidades va en `shared/`.

- Un componente exportado por archivo. Sus subcomponentes privados pueden vivir en el mismo archivo. Se acepta exportar además un auxiliar chico que usan los archivos vecinos (`EncabezadoPaso` en `PasoFecha.tsx`, `MensajeCampo` en `Alerta.tsx`).

---

14. Herramientas de Automatización

Comandos disponibles

Dónde	Comando	Qué hace
`backend/`	`npm run typecheck`	`tsc --noEmit` con `strict`
`backend/`	`npm test`	Corre toda la suite de Vitest
`backend/`	`npm run test:watch`	Vitest en modo watch (ciclo RED, GREEN, REFACTOR)
`backend/`	`npm run test:coverage`	Cobertura de los tres servicios (mínimo 90 % de líneas)
`backend/`	`npm start`	Levanta la API en `127.0.0.1:3000` con SQLite. Carga `../.env` si existe (mail real); si no, simula el mail
`frontend/`	`npm run dev`	Vite en `:5173`, con proxy de `/api` al backend
`frontend/`	`npm run build`	`tsc --noEmit` y build de producción
los dos	`npm run lint`	oxlint con las reglas de `.oxlintrc.json`
los dos	`npm run format`	Prettier corrige el formato
los dos	`npm run format:check`	Prettier verifica el formato sin escribir

Antes de abrir un PR

```
cd backend
npm run typecheck
npm run lint
npm run format:check
npm test
npm run test:coverage

cd ../frontend     # si el PR toca el frontend
npm run lint
npm run format:check
npm run build
```

Si alguno queda en rojo, el PR no se mergea.

Formato y lint

- **Prettier** (`.prettierrc.json` en la raíz) aplica el formato de la §2. Las líneas largas de `className` quedan como están, porque Prettier no parte strings.
- **oxlint** (`.oxlintrc.json` en la raíz) aplica `no-explicit-any`, `no-unused-vars` (se ignoran los parámetros que empiezan con `_`) y `eqeqeq`. Se usa en lugar de ESLint porque `typescript-eslint` todavía no soporta TypeScript 7.
- El commit que aplicó Prettier a todo el repo está en `.git-blame-ignore-revs`. Para que `git blame` lo saltee: `git config blame.ignoreRevsFile .git-blame-ignore-revs`.

---

15. Git y Ciclo TDD

Ramas

- Hay una rama por paquete de trabajo, con el formato **`tp6/pN-nombre`**, como en `tp6/p3-tarifas`.
- Los cambios que no son un paquete de TDD (frontend, configuración, documentación) usan **`tp6/<nombre>`**, sin número, como en `tp6/consistencia-estilo` (igual que `CONVENCIONES.md` §2.4).
- La integración a `main` se hace por PR o por merge, con un mensaje que dice qué se integra: el paquete (`Merge del paquete 6`) o el tema (`Alineacion de convenciones y guia de estilos`).

Alcance en el tiempo

Las reglas de esta sección rigen desde el commit `be6e48c` (20/10/2026), que agrega esta guía. Los commits y las ramas anteriores quedan como están, porque reescribir el historial de `main` sería peor que el desvío. Eso incluye las ramas `compras_service-compra_exitosa` y `pr-7`, y el formato `Test <método> <módulo> - Test N - RED|GREEN|REFACTOR` de los primeros paquetes.

Commits del ciclo (backend)

Un commit por paso, con el prefijo y el paquete. El mensaje describe el comportamiento:

Paso	Formato	Ejemplo real
RED	`red(Pn): falla …` (qué comportamiento falta)	`red(P4): falla al confirmar una compra por HTTP`
GREEN	`green(Pn): …` (qué hace ahora el sistema)	`green(P4): expone el controlador de compras`
REFACTOR	`refactor(Pn): …` (qué se mejoró sin cambiar comportamiento)	`refactor(P4): centraliza respuestas HTTP`

- **Cambios fuera del ciclo:** se usa un prefijo de tipo sin paquete. Los que hay en el historial son `config:` (herramientas y configuración), `feat:` (funcionalidad del frontend), `fix:`, `refactor:`, `style:` (solo formato), `test:` (tests sin código de producción) y `docs:`. Por ejemplo `config: centraliza ejecución local`.
- **Si un test nuevo pasa sin cambiar código:** no se commitea como RED. Se registra en un solo commit que lo aclara, como en el historial: `… [Pasa el test sin necesidad de cambiar código]` o `… YA CUBIERTO`.

Alcance de TDD

- **Backend:** todo se desarrolla con TDD. Ninguna regla se escribe sin un test que la pida antes.
- **Frontend:** no se desarrolla con TDD. Se verifica con `npm run build` y probándolo a mano contra la API.

Guía de estilo del Trabajo Práctico 6 — ISW 4K1 · Grupo 1 · 2026. Describe el código al 25/10/2026.